1. Explain the Relational Model and its components with examples.

Relational Model :

The Relational Model represents data as a collection of relations (tables). Each relation consists of rows (tuples) and columns (attributes). All data in a column must be of the same domain (data type) and each row is unique.

Components of Relational Model :

1. Relation (Table)

A relation is a table that stores data about a specific entity.

Example : EMPLOYEE relation

EmpID	EmpName	Dept	Salary
101	Ratan	HR	50000
102	Vivek	IT	60000
103	Lochan	Finance	55000
104	Nagendra	Marketing	65000

Notes :

- The table name is EMPLOYEE.
- It has 4 rows (tuples).
- It has 4 columns (attributes).

2. Tuple (Row)

A tuple is a single row in a relation. It represents one instance of an entity.

Examples :

- (101, Ratan, HR, 50000)
- (102, Vivek, IT, 60000)
- (103, Lochan, Finance, 55000)
- (104, Nagendra, Marketing, 65000)

Another Example : STUDENT

RollNo	Name	Dept	CGPA
1	Asha	CSE	8.7
2	Bharath	EEE	8.2
3	Charan	ME	7.9

Here, (1, Asha, CSE, 8.7) is a tuple.

3. Attribute (Column)

An attribute is a column in a relation. It represents a property of the entity.

Examples from EMPLOYEE relation :

- EmpID → attribute representing Employee ID
- EmpName → attribute representing Employee Name
- Dept → attribute representing Department
- Salary → attribute representing Salary

Another Example : BOOK relation

BookID	Title	Author	Price
B01	DBMS	Elmasri	650
B02	Operating Systems	Silberschatz	700
B03	Data Structures	Seymour Lipschutz	550

Attributes : BookID, Title, Author, Price

4. Domain

A domain is the set of permissible values for an attribute.

Examples from EMPLOYEE relation :

- EmpID → {101, 102, 103, 104, ...} (Integers)
- EmpName → {Ratan, Vivek, Lochan, Nagendra, ...} (Strings)
- Dept → {HR, IT, Finance, Marketing, ...} (Strings)
- Salary → {10000.00 to any higher value} (Numeric)

Another Example :

For BOOK relation

- BookID → {B01, B02, B03, ...}
- Title → {DBMS, OS, DS, ...}
- Author → {Elmasri, Silberschatz, ...}
- Price → {1 to any higher value}

5. Key

A key is an attribute or set of attributes that uniquely identifies each tuple in a relation.

Examples :

(a) EMPLOYEE relation

- Primary Key : EmpID (Value of EmpID is unique for every employee.)

(b) STUDENT relation

- Primary Key : RollNo (RollNo uniquely identifies each student.)

(c) ENROLLMENT relation (Composite Key Example)

Relation : ENROLLMENT

RollNo	CourseID	Semester
1	CS101	1
1	MA101	1
2	CS101	1

Primary Key : (RollNo, CourseID, Semester) (Combination of these uniquely identifies each tuple.)

2. Explain different types of keys in the Relational Model with suitable examples.

Keys in Relational Model:

Keys are special attributes (or set of attributes) used to uniquely identify records in a table and to establish relationships between tables.

1. Primary Key

Definition: A primary key is an attribute or a chosen set of attributes that uniquely identifies each row in a relation and cannot contain duplicate or NULL values.

Example: STUDENT

Primary Key (RollNo) →

RollNo	Name	Dept	CGPA
101	Asha	CSE	8.7
102	Bharat	EEE	8.2
103	Charan	ME	7.9

- Every row has a unique RollNo.
- No NULL values are allowed in the primary key.
- Only one primary key is chosen in a table.

---

2. Foreign Key

Definition: A foreign key is an attribute in one table that refers to the primary key of another table and is used to maintain relationships between tables.

Example:

DEPARTMENT

DeptID	DeptName
D1	CSE
D2	EEE
D3	ME

STUDENT

RollNo	Name	DeptID
101	Asha	D1
102	Bharat	D2
103	Charan	D1

Foreign Key (STUDENT. DeptID) references DEPARTMENT. DeptID

- The values in foreign key (DeptID) should match the values in the parent table (DEPARTMENT. DeptID).
- It can have repeated values.
- It creates a relationship between tables.

---

3. Alternate Key

Definition: An alternate key is a candidate key that was not selected as the primary key.

Example: EMPLOYEE

EmpID	Email	Phone	Name
E01	asha@company.com	9876543210	Asha
E02	bharat@company.com	9876543211	Bharat
E03	charan@company.com	9876543212	Charan

EmpID is selected as the Primary Key. Email and/or Phone can act as Alternate Key(s) because they are also unique.

Alternate keys also uniquely identify records.

---

4. Candidate Key

Definition: A candidate key is an attribute or minimal set of attributes that can uniquely identify each row in a table.

Example: EMPLOYEE

EmpID	Email	Phone	Name
E01	asha@company.com	9876543210	Asha
E02	bharat@company.com	9876543211	Bharat
E03	charan@company.com	9876543212	Charan

EmpID, Email, and Phone are all candidate keys because each is unique. One candidate key is chosen as the primary key and the remaining candidate keys become alternate keys.

- Candidate keys provide minimum uniqueness: no proper subset of the key can uniquely identify the rows.
- There can be one or more candidate keys in a table.

Summary: Primary Key identifies a row, Foreign Key links tables, Candidate Key is any possible unique key, Alternate Key is the candidate key not chosen as primary.

4. Explain DDL commands with syntax and examples.

DDL Commands:

DDL stands for Data Definition Language. It is used to define and modify the structure of database objects such as tables.

1. CREATE

- Used to create new database objects like tables.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype [constraints],
    column2 datatype [constraints],
    ...
);
```

Example(s):

Ex1:
```
CREATE TABLE STUDENT (
    RollNo NUMBER PRIMARY KEY,
    Name VARCHAR2(30),
    Dept VARCHAR2(20),
    Age NUMBER
);
```
Result: Creates a new table STUDENT.

Ex2:
```
CREATE TABLE DEPARTMENT (
    DeptID NUMBER PRIMARY KEY,
    DeptName VARCHAR2(30),
    Location VARCHAR2(20)
);
```
Result: Creates a new table DEPARTMENT.

2. ALTER

- Used to add, modify or drop columns in an existing table.

Syntax (Add column):
```
ALTER TABLE table_name
ADD column_name datatype
[constraints];
```

Syntax (Modify column):
```
ALTER TABLE table_name
MODIFY column_name datatype
[constraints];
```

Examples:

Ex1:
```
ALTER TABLE STUDENT
ADD CGPA NUMBER(3,1);
```
Result: Adds a new column CGPA.

Ex2:
```
ALTER TABLE STUDENT
MODIFY Name VARCHAR2(50);
```
Result: Modifies the size of Name column.

Note: ALTER can also be used to DROP columns.
Syntax: `ALTER TABLE table_name DROP COLUMN column_name;`

3. DROP

- Used to delete the entire table structure from the database.

Syntax:
```
DROP TABLE table_name;
```

Example:
```
DROP TABLE STUDENT;
```

Result: Deletes the whole table structure (data + structure).

4. TRUNCATE

- Used to remove all rows from a table, but the structure remains.

Syntax:
```
TRUNCATE TABLE table_name;
```

Example:
```
TRUNCATE TABLE STUDENT;
```

Result: Removes all rows but keeps structure.

Note: It is faster than DELETE because it deallocates storage.

5. RENAME

- Used to rename an existing table.

Syntax:
```
RENAME old_table_name TO new_table_name;
```

Example:
```
RENAME STUDENT TO STUDENT_INFO;
```

Result: Changes table name to STUDENT_INFO.

Summary of DDL Commands:

- CREATE → Creates new database objects (tables, etc.).
- ALTER → Adds, modifies or drops columns in existing table.
- DROP → Deletes the entire table structure.
- TRUNCATE → Removes all rows from a table but keeps structure.
- RENAME → Changes the name of an existing table.

Sample Table: STUDENT
RollNo (PK), Name, Dept, Age
(Used across examples)

5. Explain DML commands with syntax and examples.

DML (Data Manipulation Language) commands are used to manipulate the data present in the database tables. The main DML commands are INSERT, UPDATE and DELETE.

1. INSERT

- Used to add new rows of data into a table.
- It does not affect existing data.

Syntax :

```
INSERT INTO table_name (column1, column2, ...)
        VALUES (value1, value2, ...);
```

Example Queries :

i) Insert a single row

```
INSERT INTO EMPLOYEE (EmpID, EmpName, Dept, Salary)
VALUES (104, 'Nagendra', 'Marketing', 65000);
```

ii) Insert multiple rows

```
INSERT INTO EMPLOYEE (EmpID, EmpName, Dept, Salary)
VALUES (105, 'Asha', 'HR', 48000),
       (106, 'Bharath', 'IT', 62000);
```

Example :

Consider the EMPLOYEE table.

EmpID	EmpName	Dept	Salary
101	Ratan	HR	50000
102	Vivek	IT	60000
103	Lochan	Finance	55000

Output after above inserts :

EmpID	EmpName	Dept	Salary
101	Ratan	HR	50000
102	Vivek	IT	60000
103	Lochan	Finance	55000
104	Nagendra	Marketing	65000
105	Asha	HR	48000
106	Bharath	IT	62000

---

2. UPDATE

- Used to modify existing data in a table.
- It updates one or more rows based on a condition.

Syntax :

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Example Queries :

i) Update salary of EmpID 101

```
UPDATE EMPLOYEE
SET Salary = 55000
WHERE EmpID = 101;
```

ii) Update department of employees in IT

```
UPDATE EMPLOYEE
SET Dept = 'Information Technology'
WHERE Dept = 'IT';
```

Example :

Consider the EMPLOYEE table (from previous example).

Before Update :

EmpID	EmpName	Dept	Salary
101	Ratan	HR	50000
102	Vivek	IT	60000
103	Lochan	Finance	55000
104	Nagendra	Marketing	65000

After Update :

EmpID	EmpName	Dept	Salary
101	Ratan	HR	55000
102	Vivek	Information Technology	60000
103	Lochan	Finance	55000
104	Nagendra	Marketing	65000

(Salary of EmpID 101 updated and Dept of 'IT' employees updated.)

---

3. DELETE

- Used to remove existing rows from a table.
- It deletes rows based on a condition.

Syntax :

```
DELETE FROM table_name
WHERE condition;
```

Example Queries :

i) Delete employee with EmpID 103

```
DELETE FROM EMPLOYEE
WHERE EmpID = 103;
```

ii) Delete employees from Marketing department

```
DELETE FROM EMPLOYEE
WHERE Dept = 'Marketing';
```

Example :

Consider the EMPLOYEE table (after update).

Before Delete :

EmpID	EmpName	Dept	Salary
101	Ratan	HR	55000
102	Vivek	Information Technology	60000
103	Lochan	Finance	55000
104	Nagendra	Marketing	65000

After Delete (First query executed) :

EmpID	EmpName	Dept	Salary
101	Ratan	HR	55000
102	Vivek	Information Technology	60000
104	Nagendra	Marketing	65000

After Delete (Second query executed) :

EmpID	EmpName	Dept	Salary
101	Ratan	HR	55000
102	Vivek	Information Technology	60000

(Row with EmpID 103 deleted, then all Marketing department rows deleted.)

8. Discuss the importance of NULL values with examples.

1. NULL Values:

- NULL means absence of a value.
- It does not mean 0, blank space, or false.
- It is used when the value is unknown, missing, or not applicable.

2. Importance of NULL Values

1) Represents unknown values.

Example: In STUDENT table, PhoneNo is NULL when the number is not yet provided.

2) Represents missing information.

Example: In EMPLOYEE table, Email may be NULL if the employee has not submitted it.

3) Represents not applicable values.

Example: In PERSON table, SpouseName can be NULL for an unmarried person.

4) Prevents wrong assumptions.

- Storing 0 or blank instead of NULL can give incorrect meaning.

Example: SalaryBonus = NULL means not known yet, but 0 means no bonus.

5) Useful in optional attributes.

Example: In CUSTOMER table, AlternatePhone may be NULL because it is optional.

6) Helps in accurate database design.

Example: MiddleName can be NULL because some people do not have one.

7) Important in SQL queries.

- NULL is checked using IS NULL and IS NOT NULL, not '='.

Example Queries:

```
SELECT * FROM STUDENT WHERE PhoneNo IS NULL;
SELECT * FROM EMPLOYEE WHERE Email IS NOT NULL;
```

8) Effect on aggregate functions.

- SUM, AVG, COUNT(column) ignore NULL values, while COUNT(*) counts rows.

Example: Marks values 80, 90, NULL => AVG = 85.

3. Example Tables

STUDENT Table

RollNo	Name	PhoneNo
101	Asha	9876543210
102	Rahul	NULL
103	Meera	9123456780

EMPLOYEE Table

EmpID	Name	Email	Bonus
E01	Ravi	ravi@example.com	5000
E02	Sneha	NULL	NULL
E03	Arjun	arjun@example.com	3000

4. Important Notes

- NULL is not equal to 0.
- NULL is not equal to blank.
- Arithmetic with NULL usually gives NULL.
- Comparison with NULL uses IS NULL / IS NOT NULL.

5. Conclusion

NULL values are essential in relational databases because they represent unavailable or inapplicable data correctly. They help maintain accuracy, flexibility, and meaningful query results by distinguishing between unknown, missing, and not applicable information.